

Сервис прогнозирования отказов датчиков инженерных систем ЖКХ

Сопроводительная документация к конкурсному прототипу.

Департамент ЖКХ города Москвы · АО «Москоллектор» — направление «Отказ датчика».

Это полная документация. Краткая справка для пользователя доступна прямо в интерфейсе сервиса — раздел «Справка» после входа в систему (<https://jkh.devcosmos.ru/help>).

1. Общее описание решения

Веб-сервис прогнозирования отказов датчиков инженерных систем (насос/вентилятор, дым/газ) с диспетчерским интерфейсом, объяснимой ML-моделью и формированием заявок на превентивное обслуживание по решению диспетчера. Прототип развёрнут и доступен по адресу jkh.devcosmos.ru.

Ключевые задачи, которые решает сервис:

- Прогнозирование отказа датчика на горизонте 24 часа с указанием вероятности и объяснением.
- Приоритизация внимания диспетчера — короткий список рисков вместо потока сырых событий.
- Фиксация решений диспетчера (наблюдать / направить на проверку / отклонить / уточнить) с журналом аудита.
- Формирование заявки на обслуживание по явному решению диспетчера (автосоздание по правилу отключено — решение принимает человек).
- Аналитика и отчётность по историческим инцидентам, сезонности и заявкам на обслуживание (экспорт XLSX/PDF), реестр версий моделей с метриками.

1.1. Почему это нужно диспетчеру

Диспетчер работает с потоком срабатываний, часть которых — шум, а часть — реальный риск аварии. Цена ошибки высока в обе стороны: пропущенный отказ насоса или вентиляции — это авария и внеплановый выезд, а частые ложные тревоги — усталость персонала и потеря доверия к системе. Без явного приоритета и объяснения «почему сработал именно этот канал» диспетчер вынужден выбирать вслепую, полагаясь только на опыт.

1.2. Позиционирование решения

Решение сознательно не обещает идеального предсказания аварий — исходные данные неполные (нет координат объектов, часть физических устройств восстановлена эвристически, независимого экспертного подтверждения меток нет), и заявлять обратное было бы недобросовестно по отношению к диспетчеру, который затем столкнётся с реальностью. Вместо этого сервис берёт на себя то, что действительно решается на этих данных: приоритизацию потока событий, объяснение причины срабатывания и оформление заявки в одно действие — решение всегда остаётся за человеком. Открытое указание ограничений (см. §3.3 и §4) — не оговорка, а часть подхода: доверие к системе должно быть ровно таким, каким она его заслуживает.

Документ закрывает требования раздела 14 технического задания: используемые методы обработки данных, условия и ограничения решения, инструкции по сборке/установке, функциональную и компонентную архитектуру.

2. Функциональная и компонентная архитектура

2.1. Компоненты системы

Компонент	Технологии	Роль
backend	FastAPI, SQLAlchemy, Alembic (Python 3.12)	REST API, авторизация (JWT), бизнес-логика, миграции схемы
worker	Тот же backend-код + CatBoost, scikit-learn, pandas	Фоновый процесс: проигрывает поток исторических событий, считает признаки, вызывает модель, создаёт риск-кейсы (заявку — по решению диспетчера)
frontend	React 18 + TypeScript, Vite, Tailwind CSS	Веб-интерфейс диспетчера: дашборд, риски, журнал, заявки, реестр объектов/каналов
db	PostgreSQL 16	Единственное хранилище состояния — оперативные данные, история прогнозов и решений
caddy (только прод)	Caddy 2	Обратный прокси, автоматический TLS-сертификат (Let's Encrypt)

2.2. Поток данных

- Исторический импорт: CSV-архивы → `scripts/data/import_analysis_to_db.py` → PostgreSQL (каналы, объекты, эпизоды неисправностей, версии моделей, риск-кейсы, прогнозы).
- Живой инференс: `worker` проигрывает тот же поток событий через тот же путь обработки, что будет использовать реальный адаптер источника (CMBY) — на каждом тике считает поведенческие признаки по каналу, вызывает обученную модель (`CatBoost.predict_proba`), при превышении порога создаёт риск-кейс с SHAP-объяснением; заявка на обслуживание создаётся только явным решением диспетчера на карточке риска, автосоздания по правилу нет.
- Frontend обращается только к REST API backend'a — не имеет прямого доступа к БД. Все данные на экране — сохранённые записи, а не значения, нарисованные на клиенте.

2.3. Модель данных (основные сущности)

Object (объект, иерархия), Channel (канал датчика, привязан к объекту и, где сопоставлено, к физическому Device), IncidentEpisode (очищенный эпизод неисправности), ModelVersion, Prediction (журнал прогнозов с вероятностью и SHAP), RiskCase (открытый риск, статус, приоритет), Decision (решение диспетчера), MaintenanceRequest (заявка со сменой статусов), User/UserObjectAccess (роли и доступ по объектам), AuditLog (журнал действий).

2.4. API — ключевые эндпоинты

Полная и всегда актуальная спецификация — Swagger UI по адресу `/docs` (или `/api/docs` в зависимости от развёртывания) поднятого backend'a; ниже — ориентир по составу без деталей запрос/ответ.

Группа	Базовый путь	Основные операции
Аутентификация	<code>/api/auth</code>	POST <code>/login</code> , POST <code>/change-password</code>
Объекты	<code>/api/objects</code>	GET <code>/</code> , GET <code>/tree</code> , GET <code>/geojson</code> , GET <code>/id</code>
Каналы	<code>/api/channels</code>	GET <code>/</code> , GET <code>/id</code> , GET <code>/id/episodes</code> , GET <code>/id/degradation-trend</code>
Риск-кейсы	<code>/api/risk-cases</code>	GET <code>/</code> , GET <code>/counts</code> , GET <code>/id</code> , POST <code>/id/decisions</code>

Группа	Базовый путь	Основные операции
Прогнозы	/api/predictions	GET /, GET /{id}/summary
Заявки на обслуживание	/api/maintenance-requests	GET /, GET /{id}/history, POST /{id}/transitions, POST /{id}/approve
Модели	/api/models	GET /current, GET /
Аналитика	/api/analytics	GET /summary, GET /report.xlsx, GET /report.pdf
Дашборд	/api/dashboard	GET /summary
Доступ (только admin)	/api/access	GET/POST /users, GET/POST /users/{id}/objects, DELETE /users/{id}/objects/{object_id}
Журнал аудита (только admin)	/api/audit-log	GET /
Здоровье сервиса	/api/health	GET /live, GET /ready

Все группы, кроме auth и health, требуют JWT (Authorization: Bearer); access и audit-log дополнительно требуют роль admin.

2.5. Аутентификация и модель доступа (RBAC)

- Аутентификация — JWT (алгоритм HS256, срок жизни токена 8 часов), пароли хранятся как bcrypt-хеш; сравнение с внешними каталогами (LDAP/AD) не выполняется — см. §4.
- Роли пользователя (enum UserRole): **dispatcher** (базовая работа с рисками и заявками), **analyst** (доступ к тем же данным для анализа, без операций изменения статуса заявки), **admin** (дополнительно — управление пользователями, назначениями доступа и журналом аудита). Решения по риск-кейсам и создание заявок доступны ролям dispatcher/analyst/admin; переходы статуса и подтверждение заявки — только dispatcher/admin.
- Доступ по объектам — таблица UserObjectAccess (пара пользователь–объект). Действующее правило MVP: если у пользователя нет ни одной записи в этой таблице, ограничение не применяется и он видит все объекты — это осознанное решение для упрощённого MVP, а не пробел.
- Полная матрица доступа по организационным подразделениям (не только по отдельным объектам) не реализована — см. §4.

3. Используемые методы обработки данных

3.1. Подготовка данных

- Аудит 8 годовых архивов (2019–2026, ~313 млн событий, ~16 ГБ), манифест с SHA-256, полный профиль по годам.
- Построение очищенных эпизодов отказа (насос, вентилятор, дым, газ) двухслойным денойзингом: фильтр дневного флаппинга (>10 переходов/сутки на канал) + debounce устойчивости ≥1 час.
- Политика разметки без доступа к эксперту (файл label-policy.md) зафиксирована отдельно: целевая метка — «переход в устойчивое состояние “Неисправен”» в ближайшие 24 часа.
- Соответствие канал → физическое устройство для насос/вентилятор построено эвристически: идентификатор устройства (номер + пикет) парсится из свободнотекстового названия датчика, покрытие 87.8% каналов (469/534).

3.2. Признаки и модели

Витрина признаков на часовой сетке: окна 1ч/24ч/7сут, свежесть последнего события, соседи по расположению. На её основе обучено и задеплоено четыре модели — две основные и две дополнительные, по одной каждого типа на трек:

Модель	Тип	Тип обучения	Треки (2 экземпляра)	Роль
CatBoost	Основная	С учителем, на размеченных отказах	насос/вентилятор, дым/газ	Прогноз вероятности перехода в «Неисправен» на горизонте 24ч — центральный сигнал риска
IsolationForest	Дополнительная	Без учителя, на тех же поведенческих признаках	насос/вентилятор, дым/газ	Независимый сигнал аномальности — отмечает поведение вне размеченных сценариев отказа, не пересекается по обучению с CatBoost

- Обе основные модели (`CatBoostClassifier`) обучены с временным `train/validation/test` и разрывом 24 часа на границах — без утечки будущего в обучение.
- Базовая линия для сравнения — простая эвристика, ROC-AUC которой заведомо хуже CatBoost (0.54 против 0.71 на насос/вентилятор).
- Объяснение каждого прогноза CatBoost — SHAP-значения из самого CatBoost (`get_feature_importance, type=ShapValues`), а не отдельная библиотека.
- Поверх SHAP — необязательный слой удобства: краткое резюме прогноза на естественном языке от GigaChat-2 (Sber, напрямую, без прокси — российский сервис, доступный с прод-сервера), пересказывающее те же SHAP-факторы словами для диспетчера; результат кешируется, повторные просмотры карточки не дёргают API повторно; не подменяет и не заменяет объяснение модели, самостоятельной моделью не считается. Без настроенного ключа функция просто недоступна, остальной сервис не деградирует.
- Дополнительный небольшой сигнал вне списка моделей — тренд деградации канала: сравнение частоты «чистых» (не флаппинг) эпизодов за недавний и базовый 90-дневные периоды (`worsening/stable/improving`); это статистика по данным, а не обученная модель.

3.3. Честная оценка качества

Целевые показатели `Precision > 0.7` и `Recall > 0.5` не достигнуты ни для насос/вентилятор (ROC-AUC 0.71), ни для дым/газ (ROC-AUC 0.63) — притом что оба заметно лучше случайного угадывания (0.5) и базовой линии. На эпизодном уровне (рабочая точка порога, задеплоенная модель, тай-брейк одинаковых `timestamp` учтён): насос/вентилятор — 11,27% пойманных эпизодов (88/781) при 3,58 ложных сериях на 100 устройств/сутки (порог 0,55); дым/газ — 18,48% (202/1093) при 6,00 ложных сериях на 100 устройств/сутки (порог 0,53). Изменение по сравнению с предыдущим пересчётом небольшое (формальный статистический тест не проводился); решение — сохранить текущую модель. Три проверенные гипотезы улучшения признаков (соседние устройства в отказе, дневная гранулярность, числовые показания газовых каналов) не дали качественного прорыва.

Эти цифры измерены офлайн. Демонстрационный `worker` воспроизводит тот же поток событий, но при недостаточной истории канала или длительном отсутствии событий его расчёт расходится с офлайн-оценкой — это открытый пункт дальнейшей проверки, предупреждения на демо требуют проверки диспетчером, а не прямого приравнивания к метрикам из этого раздела.

Полный разбор с методикой — файлы `model_report_насос_вентилятор.md` и `model_report_дым_газ.md` (папка `artifacts/`). Организаторы подтвердили, что целевые метрики плановые, а не жёсткие требования, и их можно снижать при обоснованном разборе причин — рабочая точка порога выбрана по эпизодной оценке, а не по формальному 0.7/0.5.

В прежнем PDF здесь была ссылка на «Приложение А», которого в файле нет. Доступный источник ответов организаторов — CSV-выгрузка «Город 8. ДЖКХ.xlsx — Вопросы_нормализованные_и_Ответы.csv» (папка docs/source-data/).

4. Условия и ограничения решения

Полная таблица — файл «Матрица соответствия ТЗ» (Матрица_соответствия_ТЗ.md). Ниже сведены пункты, отклонённые от базового ТЗ, с обоснованием.

Пункт ТЗ	Статус	Обоснование
Интерактивная GPS-карта (раздел 10)	Заменено	Организаторы подтвердили: реальных координат объектов не будет никогда (только текущие, теряются при демонтаже датчика) — иерархическая схема объектов с цветовой индикацией риска рекомендована ими же вместо карты.
Сценарный анализ пожар/проникновение/подтопление (раздел 6)	Не реализовано	Вне объёма двухнедельного MVP; направления оцениваются организаторами независимо, жёсткого приоритета нет. Трек дым/газ частично касается пожарного риска теми же датчиками, но не заявлен как отдельное направление.
Интеграция с СМБУ, журналами ОДС, внешним реестром оборудования (разделы 6, 7, 10)	Заменено имитацией	Реального доступа нет и не будет (подтверждено организаторами) — заменено воспроизведением исторических событий через тот же путь обработки, что будет использовать реальный адаптер источника.
LDAP/AD (раздел 11)	Не реализовано	Тестового контура LDAP/AD на время мероприятия нет; локальная JWT-авторизация с ролями и матрицей доступа по объектам — принятое временное решение для MVP.
RBAC по подразделениям (раздел 11)	Частично	Реализованы роли (dispatcher/analyst/admin, см. §2.5) и доступ по объектам (UserObjectAccess); полная матрица по организационным подразделениям — задел для будущего пилота.
Push/WebSocket-уведомления (раздел 10)	Заменено периодическим опросом	Организаторы подтвердили: push/email/SMS/эскалации желательны, но не обязательны для MVP — реализовано клиентское уведомление (тост + счётчик в меню) на периодическом опросе раз в 30 секунд.
Синхронизация с внешней базой оборудования (раздел 10)	Не реализовано	Внешний реестр оборудования не передан и не будет передан организаторами. Внутреннее соответствие канал→устройство построено эвристически из собственных данных (см. раздел 3.1).

5. Инструкция по сборке, установке и запуску

5.1. Требования к серверу

- Linux (Ubuntu 22.04/24.04 LTS), Docker Engine + Docker Compose plugin.
- Рекомендуется 4 vCPU / 8 ГБ RAM / 100 ГБ SSD (минимум — 2 vCPU / 4 ГБ / 50 ГБ).
- Публичный IP и домен (для TLS через Let's Encrypt) — только для прод-развёртывания.

5.2. Локальный запуск (разработка/проверка)

```
git clone <репозиторий> jkh && cd jkh
cp .env.example .env # заполнить пароли/секреты — не коммитить
docker compose -f docker-compose.yml -f docker-compose.dev.yml up -d --build
docker compose exec backend alembic upgrade head
```

Frontend будет доступен на <http://localhost:5174>, backend API — на <http://localhost:8000/api> (Swagger UI: [/docs](http://localhost:8000/docs)).

5.3. Прод-развёртывание

```
export DOMAIN=<ваш-домен> # должен указывать A-записью на IP сервера
docker compose -f docker-compose.yml -f deploy/docker-compose.prod.yml up -d --build
docker compose exec backend alembic upgrade head
```

Caddy автоматически получает и обновляет TLS-сертификат Let's Encrypt по домену `DOMAIN`.

5.4. Тестирование и проверка качества

- Backend test suite: 107 тестов (pytest), требует отдельный тестовый Postgres — подробности см. `backend/README.md`. Отдельно ML и служебные скрипты — 7 тестов (pytest `ml/tests scripts/tests`).
- Нагрузочная проверка 20 параллельных пользователей — 0 ошибок на всех ключевых эндпоинтах (ApacheBench, запись 21 сентября — подробности см. `docs/documentation/Протокол_проверок.md`). Стенд проверки (14 ядер/48 ГБ) мощнее целевого прод-сервера (4 vCPU/8 ГБ); совместная нагрузка чтения с активным тиком worker не проверялась.
- Backup/restore: `deploy/backup.sh / deploy/restore.sh (pg_dump -Fc / pg_restore --clean)`, время в разы меньше норматива ≤4 часа; реально сработало на проде при инциденте с потерей данных пользователей.

```
docker run -d --name jkh_test_db -e POSTGRES_USER=jkh -e POSTGRES_PASSWORD=jkh \
-e POSTGRES_DB=jkh_test -p 55432:5432 postgres:16
cd backend && python3 -m pytest tests/ -v
```

6. Реестр библиотек и компонентов

Полный перечень использованных библиотек, их версий, лицензий и назначения — файл `Лицензии_и_зависимости.md` (папка `docs/documentation/`). Кратко по слоям: backend — FastAPI/SQLAlchemy/Alembic/Pydantic (все MIT), worker дополнительно — CatBoost/scikit-learn/pandas/pyarrow (Apache-2.0/BSD-3), frontend — React/TypeScript/Vite/Tailwind (MIT/Apache-2.0), инфраструктура — PostgreSQL 16, nginx, Caddy 2. Все лицензии — permissive open source, без ограничений на использование в конкурсном прототипе.

Отдельного пакета для SHAP не используется — расчёт встроен в CatBoost.

7. Нормативное соответствие

Исходный код открыт, не откомпилирован, не обфусцирован. Работа с персональными данными ограничена собственными учётными записями пользователей сервиса (логин, роль, журнал действий) — пароли хранятся в виде bcrypt-хеша, передача данных — по TLS 1.2+. Исходные данные конкурса обезличены на стороне организаторов до передачи. Соответствует требованиям 149-ФЗ и 152-ФЗ в объёме, применимом к прототипу такого масштаба.

8. Глоссарий терминов решения

Термины из раздела 1 технического задания здесь не дублируются — только специфичные для реализации, без которых остальные разделы документа не читаются однозначно.

Термин	Определение
Эпизод (очищенный эпизод)	Run-length интервал состояния «Неисправен» по каналу, прошедший фильтры дребезга и дневного флаппинга (см. ниже) — единица подсчёта в эпизодной оценке качества (§3.3), не то же самое, что «строка обучающей выборки».
Дребезг	Короткие (секунды) переходы канала в «Неисправен» и обратно, не связанные с реальным отказом; 70–90% сырых переходов — дребезг, а не отказ. Отфильтровывается debounce-порогом.
Debounce (порог устойчивости)	Правило: интервал состояния «Неисправен» засчитывается как эпизод только при длительности ≥ 1 час (либо не восстановился до конца окна наблюдения).
Дневной флаппинг	Канал с более чем 10 переходами в «Неисправен» за календарные сутки помечается как технический инцидент этого дня; такие переходы не считаются самостоятельными отказами.
Левостензурированный эпизод	Эпизод, чья первая запись в наблюдаемом окне уже в состоянии «Неисправен» — начало перехода не видно в данных, вероятно продолжение более раннего отказа. Исключается из численного сравнения offline/runtime.
Целевая переменная / метка	Косвенный признак — переход канала в устойчивое состояние «Неисправен» в ближайшие 24 часа. Это не подтверждённая физическая поломка, а зафиксированный по данным CMBU переход состояния.
in_fault_now	Runtime-приближение «канал уже в отказе прямо сейчас» (используется worker'ом, у которого, в отличие от офлайн-расчёта, нет доступа к будущему восстановлению эпизода) — не открывает новый риск-кейс, пока канал в этом состоянии.
Offline / runtime (worker)	Offline — расчёт признаков и метрик на всей истории сразу (ml/features/build_features.py, отчёты моделей); runtime/worker — тот же расчёт по тикю, без доступа к будущему, в демонстрационном сервисе. Числа могут расходиться, особенно для каналов с длительным молчанием — см. §3.3.
Риск-кейс (RiskCase)	Открытая карточка риска по каналу — создаётся при превышении рабочего порога модели, содержит прогноз, SHAP-объяснение и статус; закрывается решением диспетчера или автоматически по таймауту без новых предупреждений.
Тай-брейк	Правило устранения неоднозначности при одинаковом timestamp у нескольких точек данных (например, порядок вытеснения при retention) — источник части расхождений между версиями пересчёта метрик.

Термин	Определение
ROC-AUC / PR-AUC	Стандартные метрики качества бинарного классификатора: ROC-AUC — способность модели ранжировать положительные примеры выше отрицательных (0.5 — случайное угадывание); PR-AUC — то же в терминах Precision/Recall, устойчивее к сильному дисбалансу классов (как в этой задаче).
SHAP	Метод объяснения прогноза через вклад каждого признака в конкретное предсказание; здесь считается встроенными средствами CatBoost, отдельная библиотека не используется.

9. Итоговые преимущества решения

- **Честная оценка на языке диспетчера.** Качество измерено не только оконными Precision/Recall, но и на эпизодном уровне (§3.3) — в тех единицах, которые реально важны для работы: сколько эпизодов поймано и сколько ложных серий это стоит.
- **Объяснимость вместо чёрного ящика.** Каждый прогноз сопровождается SHAP-разбором вклада признаков (§3.2), а не только числом вероятности — диспетчер видит, почему сработал риск, а не просто факт срабатывания.
- **Один рабочий сценарий вместо разрозненных систем.** Риск → объяснение → решение → заявка → журнал — без переключения между инструментами (§1, §2.2).
- **Открытый код на permissive open-source стеке.** Без обфускации, без вендор-лока — весь стек (§6) можно развивать и поддерживать любой командой.
- **Готовая модель доступа и аудита.** RBAC с ролями и матрицей доступа по объектам, полный журнал решений диспетчера из коробки (§2.5) — не заглушка, а рабочий механизм.
- **Проверенное, а не только заявленное восстановление данных.** Backup/restore не только описаны, но и реально использовались на практике при инциденте (§5.4) — это отличает решение от многих прототипов, где резервное копирование существует только в документации.